传智杯 AI 创新应用大赛 · 参赛技术文档

智汇校园 · 二手集市

AI 驱动的可信校园闲置交易平台

本地 AI 引擎 · 零外部 API · 断网可用 · 24 项冒烟测试全通过 · 15 次细粒度提交

作品名称	智汇校园 · 二手集市（campus-secondhand）
参赛同学	（填写姓名 / 学号）
指导教师	（填写）
学校	（填写）
仓库地址	https://github.com/1852290xc/AI-
版本	v1.0.0 ｜ 2026 年 10 月

文档范围

1 摘要与定位　4 意图控制策略　7 评测与结果
2 场景与问题　5 本地 AI 引擎设计　8 演进记录
3 系统架构　6 可复现性论证　9 局限与后续
附录 A 数据模型字段　附录 B 接口清单

1　摘要与作品定位

“智汇校园 · 二手集市”是一个面向高校的轻量闲置交易平台。它要解决的不是“再做一个二手网站”，而是校园二手交易里最劝退的一步：把一件闲置**说清楚、定好价、发出去**。

学生在发布时往往卡在三件事上——标题不会写、描述不知道写什么、价格拿不准。平台把这三件事交给一组运行在本地的 AI 能力：输入标题后实时判断品类、按成色给出定价区间、按品类生成描述文案，同时在提交前做一次违规词审核，命中即拦截并说明原因。

七项 AI 能力全部用纯 Python 规则引擎实现：关键词打分、成色折价表、模板渲染、违规词库、标签抽取、三因子加权推荐、问答库匹配。它们不调用任何外部大模型接口，因此有三个直接好处：**零调用成本、毫秒级响应、断网可用**——答辩现场即使没有网络，所有 AI 演示照样跑得起来。

工程上，项目采用 Flask 应用工厂 + 七个功能蓝图的结构，SQLite 单文件存储，前端为原生 CSS/JS 不依赖构建工具。配套 24 项冒烟测试，覆盖游客访问、注册登录、六个 AI 接口、发布与审核、收藏、下单约束、私信审核、后台权限与退出登录；当前结果 24/24 全部通过。

表 1-1 交付概览

交付维度	结果
功能完整度	注册登录、发布/编辑/下架/删除、收藏、下单与订单流转、私信、AI 助手、管理后台，共 7 个蓝图
AI 能力	分类 / 定价 / 文案 / 审核 / 标签 / 推荐 / 客服，7 项全部本地实现
可验证性	24 项冒烟测试，独立测试库，可一键重跑
可复现性	三条命令启动；依赖版本锁定；数据由脚本确定性生成
工程规范	15 次细粒度提交，对应 5 个开发阶段；MIT 协议
代码规模	Python 约 1 220 行、模板 934 行、样式与脚本 607 行

2　场景与问题

2.1　校园二手的三个摩擦点

在校园里观察一圈会发现：闲置不是卖不掉，而是**发不出去**。

- 发布成本高：一件 79 元的静音鼠标，写标题、想描述、拍照、定价，前后要十几分钟；很多人算了一下，直接放弃了。

- 价格无参照：学生的定价依据大多是“我当时多少钱买的”，缺少按成色折算的常识，挂高了没人问，挂低了心里不舒服。
- 内容无审核：校园群里常见代考、外挂、代写论文一类信息，既污染环境，也让真实交易更难被信任。

2.2 目标用户与核心任务

目标用户是本科生与研究生，典型场景是学期初采购、学期末清仓、毕业季甩卖。核心任务只有一个：**在三分钟内把一件闲置挂到平台上，并且挂得像回事**。围绕这个任务，产品的每个设计都指向“减少输入”：分类可以留空交给 AI、描述可以一键采纳、价格有区间参考、违规内容在提交前就被挡住。

2.3 与既有方案的差异

表 2-1 与既有方案的差异

对比项	校园群 / 论坛	通用二手平台	本作品
发布门槛	会写就行，但没人给建议	表单复杂，定价靠自己	AI 补分类、文案、价格
内容治理	基本没有	平台侧强审核	本地词库，提交前拦截
推荐	纯时间序	商业算法，不可解释	三因子加权，可解释、可复现
运行成本	0	广告/服务费	0，且断网可用

3 系统架构

3.1 分层与职责

整体是四层结构，层间只通过明确的数据结构通信，任何一层都可以单独替换。

表 3-1 四层职责

层	组成	职责	关键约束
表现层	Jinja2 模板 21 个 + style.css + main.js	渲染页面、表单、异步交互	不写业务判断，只消费接口返回
接口层	7 个蓝图（main/auth/product/order/message/ai/admin）	路由、参数校验、权限、响应	每个蓝图只管一个领域
领域层	models.py 的 7 张表 + ai_engine.py 的 7 个函数	数据结构与规则计算	AI 函数必须是纯函数，可直接单测
存储层	SQLite（data/campus_market.db）	持久化、外键约束	单文件，零安装

3.2 应用工厂与蓝图边界

app.py 采用工厂函数 create_app(config_class)，把配置作为参数注入。这样测试可以直接传入 TestConfig，把数据库换成独立的 test_smoke.db，而不必改动生产配置——这也是第 7 章“测试不污染演示数据”的前提。

蓝图注册集中在 blueprints/__init__.py 的 register_blueprints(app) 中，app.py 只调用这一个函数。新增模块时，改动被限制在两处（新蓝图文件 + 注册表），不会扩散到入口文件。

```
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)          # 会话与登录态
    register_blueprints(app)             # 7 个蓝图统一挂载
    @app.context_processor               # 全站共享站点名与分类
    def inject_globals(): ...
    @app.errorhandler(404)               # 统一错误页
    def not_found(error): ...
    return app
```

3.3 数据模型

七张表覆盖“人—货—交易—沟通—行为”五个对象。所有表都带 id、created_at、updated_at；密码只存 werkzeug 生成的哈希，不存明文。

表 3-2 数据模型

表	存什么	关键字段	关系
User	学生 / 管理员	student_id、nickname、email、password_hash、school、credit、is_admin、banned	1–N Product
Category	商品分类	name、icon、sort	1–N Product
Product	闲置商品	title、description、price、original_price、condition、image、ai_tags、status、views	N–1 User/Category
Favorite	收藏关系	user_id、product_id	唯一约束 (user, product)
Order	订单	buyer_id、seller_id、product_id、amount、status	N–1 三方
Message	私信	sender_id、receiver_id、content、is_read	N–1 User
Behavior	行为记录	user_id、product_id、action(view/fav/order)	推荐算法输入

3.4 三条关键请求流

发布流（最核心）——它把“审核、分类、打标、落库”串成一条顺序明确的链，任一环节失败都不会落库：

```
POST /product/publish
  1. 校验：标题 ≥3 字、价格 >0、价格可转 float
```

```
  2. 审核：moderate_text(标题 + 描述) → 不通过则回填表单 + flash 原因，直接 return
  3. 分类：用未分类       classify_product() 判品类，按名称查 Category
  4. 打标：extract_tags() 生成 ai_tags（逗号分隔）
  5. 落库：save_image() 用 secure_filename + uuid 重命名，写入 Product
  6. 反馈：自动分类时提示「AI 已自动归类为 X」
```

下单流——本质是一个受约束的状态迁移，视频里“商品不能被买两次”就靠它保证：

```
POST /order/create/<pid>
  商品.status != onsale      → 拒绝（已售/已下架）
  商品.seller_id == 当前用户 → 拒绝（不能买自己的）
  已存在 pending/done 订单   → 拒绝（不能重复下单）
  通 →   建Order(pending) + 商品.status = sold + 记录 Behavior
```

私信流——发送前同样过审核，把“聊天里骂人”也纳入治理：

```
POST /message/send  {to, content}
  content 为空或 >500 字 → 400
  moderate_text(content) 不通 → 200 {ok:false, blocked:true, message}
  通过 → 落库 → 返回消息体，前端追加气泡
```

3.5 权限与安全

- 两层守卫：蓝图级 @login_required 拦截未登录；后台 before_request 再校验 is_admin，普通用户访问直接 403。
- 越权防护：编辑/删除商品时校验 seller_id == 当前用户或管理员，否则 403。
- 上传安全：secure_filename + uuid 重命名 + 扩展名白名单（png/jpg/jpeg/gif/webp）+ 5MB 上限（413 友好页）。
- 注入防护：全站使用 SQLAlchemy ORM 参数化查询，模板默认开启自动转义。
- 会话安全：SECRET_KEY 支持环境变量覆盖，README 明确提示生产环境必须替换。

4 意图控制策略

这一章回答一个问题：当代码主要由 AI 生成时，人如何确保产出“是想要的东西”，而不是“看起来像的东西”。我们的做法可以概括为四句话：**上下文分层、约束可验证、接口先行、以测试作为完成定义**。

4.1 上下文分层

每次向 AI 下达任务，输入都按四层组织，缺一层就容易跑偏。层与层之间用显式标题分隔，避免模型把示例当成需求。

表 4-1 上下文四层

层	内容	反例（会导致什么）
L1 目标	这次要交付的可运行结果，一句话说清	只写“写个二手平台” → 生成大而全的骨架
L2 约束	技术栈、文件边界、命名、禁止事项	不写“不要调用外部 API” → 引入在线服务
L3 数据契约	表名、字段、类型、返回值结构	不写返回结构 → 前端与接口对不上
L4 验收	可执行的检查项（测试/页面状态）	不写验收 → “看起来对了”但跑不通

4.2 Prompt 结构模板

实际使用的模板如下（节选自发布页 AI 辅助模块的开发指令）。它的特点是：把“不要做什么”写得比“要做什么”更具体。

```
【目标】实现发布页的实时 AI 辅助，键盘输入后 300ms 内更新四个提示条。
【技术约束】
  - 只用原生 JS，不引入框架；文件放 static/js/main.js
  - 请求走 POST /ai/*，返回 JSON；不得整页刷新
  - 输入防抖 300ms；请求失败静默，不打断用户
【数据契约】
  /ai/classify → {category: str, confidence: float, scores: dict, reason: str}
  /ai/price    → {low: float, high: float, suggest: float, discount: float, condition: str}
【验收】
  1. 输入“九成新 iPhone 13”后，分类条显示“数码电子 · 置信度 xx%”
  2. 原价填 5999、成色选九成新，定价条出现区间且 low<high
  3. 输入“外挂”时出现红色拦截提示，且隐藏其他提示
```

4.3 约束 → 断言的映射

每条约束都尽量落到一个可执行的断言上；没有断言的约束等于没有约束。这是本项目“完成”的定义方式。

表 4-2 约束与断言

约束（自然语言）	对应断言（可执行）	落点
违规内容不能发布	发布后商品数不变，且页面出现“拦截”	test_smoke #14
不能买自己的商品	响应包含“不能购买自己”	test_smoke #18
不能重复下单	第二次下单被拒且状态不变	test_smoke #19
普通用户不能进后台	GET /admin/ 返回 403	test_smoke #22
AI 分类要判对	classify('九成新 iPhone 13') == 数码电子	test_smoke #9
定价区间合理	low < suggest < high	test_smoke #10

4.4 四次典型修正（真实现象与处理）

下面四次都是开发过程中真实发生的问题。记录它们不是为了展示“AI 会犯错”，而是说明**人是如何定位并收口的**。

表 4-3 四次典型修正

序	现象	定位	处理与防回归
①	发布页渲染结构错乱：右侧栏内容被吞进表单里	模板里 <aside> 被写成了 </section>，浏览器容错导致嵌套错位	修正闭合标签；新增页面遍历脚本，逐页 GET 检查是否出现 5xx
②	接口告警 LegacyAPIWarning，未来版本将不可用	SQLAlchemy 2.x 弃用 Query.get()，代码仍用旧写法	统一替换为 db.session.get(Model, id)，并在全仓搜索残留
③	冒烟测试第 20 项失败：找不到收件人	测试假设学号为 2021002，演示数据里实际是 lisi，契约不一致	以“演示账号可登录”为准改测试与种子数据，二者对齐后通过
④	启动即报 datetime.utcnow 弃用警告	Python 3.12 起 utcnow() 不再推荐	抽出 utcnow() 辅助函数（返回朴素 UTC），全仓替换默认值

4.5 失败模式清单

同类项目最容易踩的坑，我们在过程中都遇到过，处理方式收敛成下表，作为后续迭代的检查表。

表 4-4 失败模式与对策

失败模式	表现	对策
一次性生成过大	一次要求“写完全部模板”，产出 20 个文件里必然有结构错误	按模块分批，每批不超过 3 个文件，产出即验证
隐式全局状态	跨模块直接改全局变量，测试之间互相污染	全部走 Flask 上下文；测试用独立配置与独立库
静默吞异常	try/except 后什么都不做，问题被藏起来	只在“不影响主流程”的位置静默（如 AI 提示失败），其余抛出
契约漂移	接口返回字段与前端约定不一致	先定数据结构再写实现，字段名写进验收项

4.6 “接口先行”的三段式

对一次生成容易失控的模块（如发布页、后台表格页），我们要求 AI 先只产出接口，再逐个实现：

```
第 1 段：只输出接口清单
    blueprints/product.py  —— publish() detail() edit() delete() my() favorite()
    每个函数：路由、入参、返回、权限要求、失败分支
第 2 段：人确认接口后，逐个函数实现（一次只改一个函数）
第 3 段：每完成一个函数，跑一次冒烟测试，绿了再继续
```

好处是可回退：任意一次生成出错，最多影响一个函数，git 里也是一次独立提交，便于定位与撤销。

5 本地 AI 引擎设计

5.1 七项能力

表 5-1 七项 AI 能力

#	能力	输入 → 输出	算法要点
1	智能分类	标题, 描述 → {category, confidence, scores, reason}	6 个品类关键词表；命中数归一化后映射到 0.55–0.98 置信度
2	智能定价	原价, 成色 → {low, high, suggest, discount}	成色折价表：全新 0.85 / 九成 0.70 / 八成 0.55 / 七成 0.40 / 有瑕疵 0.25；区间 ±12%
3	文案生成	标题, 品类, 成色, 原价 → 文案	6 套品类模板 + 成色话术 + 参考价拼接
4	内容审核	文本 → {passed, reason, message, hits}	4 类违规词库（脏话/违法物品/违规交易/色情低俗），首个命中即返回
5	标签提取	标题, 描述, 品类 → [tag]	品类 + 成色 + 年份型号 + 英文型号 + 品类词，去重后取前 8
6	个性化推荐	用户 → [Product]	内容相似 50% + 分类偏好 35% + 热度 15%；无行为则退化为热门
7	智能客服	问题 → {answer, matched}	6 组问答关键词，命中即答，未命中转人工

5.2 推荐算法的权重设计

推荐是唯一涉及多因子的能力，权重写死并公开，便于复现与解释：

```
score(p) = 0.50 · overlap(p.tags, 用户历史标签)
         + 0.35 · 分类偏好占比（该用户浏览该类的次数 / 总次数）
         + 0.15 · 热度（p.views / 最大 views）


冷启动：无行为记录 → 直接按 views 倒序（热门），并在页面标题上区分“为你推荐 / 热门商品”
```

5.3 可解释性

三个会“影响用户决策”的能力都返回解释字段：分类返回 `reason` 与各品类得分；定价返回采用了哪档折扣；审核返回命中的具体词与违规类别。这样用户在页面上看到的不是一个黑盒结论，而是“为什么”。

5.4 取舍：为什么不接大模型

表 5-2 技术取舍

维度	本地规则引擎（本作品）	调用大模型 API
单次成本	0	按 token 计费，发布量越大越贵
响应	毫秒级，可随输入实时刷新	数百毫秒到数秒，需防抖与排队
离线	完全可用	断网即失效
可复现	同一输入必得同一输出	受温度与模型版本影响
能力上限	不会“创作”，模板与词表覆盖有限	可生成更自然的文案
结论	选择可控与可复现，把“生成”的短板交给模板与用户编辑弥补	适合作为后续可选增强

6 可复现性论证

6.1 环境与依赖

开发与验证环境为 Windows 11 + Python 3.12.14。依赖版本全部锁定，不使用 range 约束：

表 6-1 依赖锁定

依赖	锁定版本	用途
Flask	3.1.3	Web 框架（应用工厂、蓝图、上下文）
Flask-SQLAlchemy	3.1.1	ORM 与会话管理
Flask-Login	0.6.3	登录态与 @login_required
Werkzeug	3.1.9	密码哈希、secure_filename
SQLAlchemy	2.1.6	底层 ORM（随 Flask-SQLAlchemy 引入）

6.2 数据确定性

演示数据由 init_db.py 以固定内容生成：6 个分类、3 个账号、16 件商品、5 条浏览行为，不依赖随机数、不依赖外部数据源。任何人执行 python init_db.py 得到的是同一套可演示状态（唯一不同的是自增 id 与时间戳）。

6.3 随机性说明

全仓唯一的随机来源是上传图片的文件名（uuid 前 16 位），它不影响任何业务逻辑与页面内容。AI 引擎全部为纯函数：无随机、无时间依赖、无网络调用，因此同一输入在任何机器上输出一致。

6.4 三条命令复现

```
pip install -r requirements.txt
python init_db.py      # 建表 + 6 分类 + 3 账号 + 16 商品 + 5 条行为
python app.py          # http://127.0.0.1:5000
```

验证复现是否成功，看三个信号：命令行出现「✅ 数据库初始化完成」；首页能看到 16 件商品；用 zhangsan/123456 登录后首页标题变为「为你推荐」。

6.5 环境差异与已知边界

- 数据库文件路径由 config.py 基于 __file__ 推导，不写死绝对路径，Windows / macOS / Linux 均可直接运行。
- SQLite 为 Python 内置，不存在“装了不同版本数据库”的差异；唯一差异是不同 Python 小版本的 SQLite 略有不同，不影响本项目的表结构。

- 前端不依赖 CDN，Bootstrap 为可选项（static/vendor 留空也能正常显示，样式由 style.css 提供）。
- 未验证：Windows 以外的中文路径兼容性；高并发下的写冲突（SQLite 单写者模型）。

7 评测与结果

7.1 测试设计

test_smoke.py 使用 Flask 测试客户端模拟浏览器，在独立数据库 data/test_smoke.db 上运行，不会污染演示数据。24 项按五组组织：

表 7-1 测试分组

组	覆盖点	项数
游客	首页、分类筛选、关键词搜索、关于页	4
认证	注册成功、重复注册被拒、登录成功、错误密码被拒	4
AI 接口	分类、定价区间、客服命中、审核拦截	4
业务	发布+自动分类、违规拦截、收藏/取消、下单、买自己的、重复下单、私信、私信拦截	9
权限	普通用户进后台 403、管理员进后台、退出登录	3

7.2 结果

当前版本运行 python test_smoke.py 输出：

```
部通  过！24/24
```

除自动化测试外，另做了一次页面级巡检：以登录态逐个访问 15 个页面，全部返回 200（后台在普通用户下为 403，符合预期），未出现 5xx。真实启动（python app.py）后，/api/health 返回 200，首页渲染出 16 件商品。

7.3 未覆盖与风险

- 并发：未做多用户同时下单的竞态测试，当前依靠 status 字段的先后写入，理论上存在极短窗口的重复售出风险。
- 上传：仅校验扩展名与大小，未校验真实文件类型（可通过魔数增强）。
- 邮箱验证：注册未做邮件验证，仅在表单层校验格式。
- 推荐冷启动：新用户完全依赖热度，存在“越热越热”的马太效应。

8 演进记录

8.1 提交与阶段

仓库保留 15 次细粒度提交，与五个开发阶段一一对应。每次提交都能独立说明“这次加了什么”。

表 8-1 十五次提交与阶段映射

阶段	提交	内容
一、初始化	1	目录结构、.gitignore、requirements、README 骨架
二、后端核心	3	配置与 7 张模型 / 本地 AI 引擎 / 应用工厂
三、功能接口	3	首页与认证 / 商品·订单·私信 / AI 接口与后台
四、前端页面	5	公共骨架与首页 / 认证页 / 商品页 / 订单·私信·后台 / 样式脚本
五、数据与发布	3	初始化脚本与演示数据 / 24 项冒烟测试 / README 与 MIT 协议

8.2 三个关键决策

为什么用 SQLite。 比赛与答辩场景最重要的是“别人拿到就能跑”。SQLite 是 Python 内置，省掉安装与配置数据库的一整段说明，也让备份变成“拷一个文件”。代价是写入并发有限，已在第 7.3 节列为已知风险。

为什么 AI 不接大模型。 三个理由按重要性排序：断网可用（答辩现场网络不可控）、可复现（同一输入同一输出，便于写测试）、零成本。代价是生成能力弱，用模板与一键采纳来弥补。

为什么前端不用框架。 页面数量有限（21 个模板），交互集中在发布页与聊天页；原生 JS 让“查看源码即可理解”这件事成立，也避免了构建工具带来的复现成本。

9 局限与后续

当前版本是一个“可完整演示、可一键复现”的 1.0，仍有明确的边界：

- AI 生成能力弱：文案是模板拼接，缺少个性化表达；后续可引入可选的本地小模型作为增强，同时保留规则引擎作为兜底。
- 审核靠词表：对变体、谐音、拼音缩写覆盖不足，需要维护词库或引入相似度匹配。
- 并发与一致性：下单与库存的状态迁移缺少事务级锁，规模化前需引入乐观锁（版本号）。
- 评测缺真实用户：当前指标以功能正确性为主，缺少真实用户的任务完成率与满意度数据。

下一步优先级：① 下单加乐观锁；② 上传做文件类型校验；③ 发布页支持图片自动打标（本地模型）；④ 增加真实用户小样本评测，作为产品迭代依据。

附录 A　数据模型字段

表 A-1 字段清单

表	字段（类型）
User	id·student_id(str32,uniq)·nickname(str32)·email(str120,uniq)·password_hash(str255)·school(str64)·credit(int)·is_admin(bool)·banned(bool)·created_at·updated_at
Category	id·name(str32,uniq)·icon(str8)·sort(int)·created_at·updated_at
Product	id·title(str120)·description(text)·price(float)·original_price(float)·condition(str16)·image(str255)·ai_tags(str255)·status(str16: onsale/sold/off)·views(int)·category_id(FK)·seller_id(FK)·时间戳
Favorite	id·user_id(FK)·product_id(FK)·时间戳；唯一约束 (user_id, product_id)
Order	id·buyer_id(FK)·seller_id(FK)·product_id(FK)·amount(float)·status(str16: pending/done/canceled)·时间戳
Message	id·sender_id(FK)·receiver_id(FK)·content(text)·is_read(bool)·时间戳
Behavior	id·user_id(FK)·product_id(FK)·action(str16: view/fav/order)·时间戳

附录 B　接口清单

表 B-1 路由清单

方法	路径	说明	权限
GET	/	首页（推荐/热门、分类、搜索、分页）	公开
GET	/about	关于页	公开
GET	/api/health	健康检查	公开
GET/POST	/auth/register	注册	公开
GET/POST	/auth/login	登录	公开
GET	/auth/logout	退出	登录
GET	/auth/profile	个人中心	登录
POST	/auth/profile/edit	修改资料	登录
GET/POST	/product/publish	发布（AI 审核 + 自动分类）	登录
GET	/product/<id>	商品详情	公开
GET/POST	/product/<id>/edit	编辑商品	本人/管理员
POST	/product/<id>/delete	删除商品	本人/管理员
POST	/product/<id>/off	上/下架	本人/管理员
GET	/product/my	我的商品	登录
POST	/product/<id>/favorite	收藏切换（JSON）	登录
POST	/order/create/<pid>	下单	登录

方法	路径	说明	权限
GET	/order/mine	我的订单	登录
POST	/order/<id>/done	确认完成	买卖双方
POST	/order/<id>/cancel	取消订单	买卖双方
GET	/message/inbox	收件箱	登录
GET	/message/chat/<uid>	聊天页	登录
POST	/message/send	发送（AI 审核，JSON）	登录
GET	/message/poll/<uid>	轮询新消息	登录
GET	/ai/assistant	AI 助手页	公开
POST	/ai/classify·/ai/price·/ai/copy·/ai/moderate·/ai/tags·/ai/chat	六个 AI JSON 接口	公开
GET	/ai/recommend	推荐结果（JSON）	公开
GET	/admin/	后台看板	管理员
GET	/admin/products	商品管理	管理员
POST	/admin/product/<id>/<action>	上/下架、删除	管理员
GET	/admin/users	用户管理	管理员
POST	/admin/user/<id>/<action>	封禁/解封/授权	管理员
GET/POST	/admin/categories	分类管理	管理员

本文档所述功能、测试结果与提交记录，均可在仓库 https://github.com/1852290xc/AI- 复现。